

ASSIGNMENT: 8

1. Write a Program to perform the following operations on the array:

Algorithm:

1. Start
2. Input size and elements of array
3. Pass array and size to a function by reference
4. In the function, multiply each element by 2
5. Return to main
6. Print modified array
7. Stop

Code:

```
#include <stdio.h>

void multiplyByTwo(int *arr, int n) {
    for(int i = 0; i < n; i++) {
        arr[i] *= 2;
    }
}

int main() {
    int arr[10], n;
    printf("Enter number of elements: ");
    scanf("%d", &n);

    printf("Enter elements: ");
    for(int i = 0; i < n; i++)
        scanf("%d", &arr[i]);
```

```

multiplyByTwo(arr, n);

printf("Modified array: ");
for(int i = 0; i < n; i++)
    printf("%d ", arr[i]);
return 0;
}

```

2. Write a C program that searches for an element in an array using a user defined function and pointers.

Input: Enter elements: 10 20 30 40 50 60

Output: Enter element to search: 30

Element found at index: 2

Algorithm:

1. Start
2. Input array and element to search
3. Pass array, size, and target to a function using pointers
4. In function, loop through and compare
5. Return index if found
6. Else return -1
7. Print result
8. Stop

Code:

```

#include <stdio.h>

int searchElement(int *arr, int n, int key) {
    for(int i = 0; i < n; i++) {
        if(*(arr + i) == key)
            return i;
    }
    return -1;
}

```

```
}
```

```
int main() {  
    int arr[100], n, key;  
    printf("Enter number of elements: ");  
    scanf("%d", &n);  
  
    printf("Enter elements: ");  
    for(int i = 0; i < n; i++)  
        scanf("%d", &arr[i]);  
  
    printf("Enter element to search: ");  
    scanf("%d", &key);  
  
    int result = searchElement(arr, n, key);  
    if(result != -1)  
        printf("Element found at index: %d\n", result);  
    else  
        printf("Element not found\n");  
  
    return 0;  
}
```

3. Write a C program that calculates the factorial of a number using a function.

Algorithm:

1. Start
2. Input number
3. Call factorial function

4. Calculate factorial using loop
5. Return result
6. Print factorial
7. Stop

Code:

```
#include <stdio.h>
```

```
int factorial(int n) {  
    int fact = 1;  
    for(int i = 1; i <= n; i++)  
        fact *= i;  
    return fact;  
}
```

```
int main() {  
    int n;  
    printf("Enter a number: ");  
    scanf("%d", &n);  
  
    printf("Factorial: %d\n", factorial(n));  
    return 0;  
}
```

4. Write a C program to remove spaces in a string using pointers.

Algorithm:

1. Start
2. Input string
3. Use pointer to copy non-space characters
4. Print modified string
5. Stop

Code:

```

#include <stdio.h>

void removeSpaces(char *str) {
    char *p = str, *q = str;
    while(*p) {
        if(*p != ' ')
            *q++ = *p;
        p++;
    }
    *q = '\0';
}

int main() {
    char str[100];
    printf("Enter a string: ");
    fgets(str, sizeof(str), stdin);

    removeSpaces(str);
    printf("String without spaces: %s\n", str);
    return 0;
}

```

5. Write a C program to replace a character in a string by another character using pointers

Test case:

Welcome to VIT

O/p: Welcxme tx VIT

Algorithm:

1. Start
2. Input string, character to replace, and replacement
3. Traverse string using pointer
4. Replace where character matches
5. Print modified string
6. Stop

Code:

```
#include <stdio.h>

void replaceChar(char *str, char oldChar, char newChar) {
    while(*str) {
        if(*str == oldChar)
            *str = newChar;
        str++;
    }
}

int main() {
    char str[100];
    printf("Enter a string: ");
    fgets(str, sizeof(str), stdin);

    replaceChar(str, 'o', 'x');
    printf("Modified string: %s\n", str);
    return 0;
}
```

6. Write a program to create a user defined function to join two strings without using Library function and print concatenated strings on the screen.

String1: Ram

String2: Sharma

String1: Ram Sharma

Algorithm:

1. Start
2. Input both strings
3. Use a loop to reach end of first string
4. Append second string character by character
5. Add null terminator
6. Print result
7. Stop

Code:

```
#include <stdio.h>
```

```
void concat(char *s1, char *s2) {  
    while(*s1) s1++;  
    *s1++ = ' '; // add space  
    while(*s2)  
        *s1++ = *s2++;  
    *s1 = '\0';  
}
```

```
int main() {  
    char s1[100] = "Ram";  
    char s2[] = "Sharma";  
  
    concat(s1, s2);  
    printf("String1: %s\n", s1);  
    return 0;
```

```
}
```

7. Write a program to create a user defined function for finding length of String and print on the screen without using library function.

Algorithm:

1. Start
2. Input string
3. Count till null terminator
4. Print count
5. Stop

Code:

```
#include <stdio.h>
```

```
int stringLength(char *str) {  
    int len = 0;  
    while(*str++) len++;  
    return len;  
}
```

```
int main() {  
    char str[100];  
    printf("Enter a string: ");  
    fgets(str, sizeof(str), stdin);  
  
    printf("Length: %d\n", stringLength(str));  
    return 0;  
}
```


8. Write a program to compare the contents of two strings whether strings are equal or not and print appropriate messages on the screen without using the library function.

Algorithm:

1. Start
2. Input two strings
3. Compare char by char
4. If mismatch, not equal
5. If both end, equal
6. Stop

Code:

```
#include <stdio.h>

int compareStrings(char *s1, char *s2) {
    while(*s1 && *s2) {
        if(*s1 != *s2)
            return 0;
        s1++; s2++;
    }
    return (*s1 == '\0' && *s2 == '\0');
}

int main() {
    char s1[100], s2[100];
    printf("Enter first string: ");
    fgets(s1, sizeof(s1), stdin);
    printf("Enter second string: ");
    fgets(s2, sizeof(s2), stdin);

    if(compareStrings(s1, s2))
        printf("Strings are equal.\n");
}
```

```

else
    printf("Strings are not equal.\n");
return 0;
}

```

9. Given a string s, perform following operations on a string using function (Without pointer):

- a. Find a length of a string.**
- b. Print string in reverse order.**
- c. Copy string s into s1 and print it.**
- d. Accept another string, say s2. Concatenate s and s2 and print concatenated string.**

Code:

```

#include <stdio.h>

int strLength(char str[]) {
    int i = 0;
    while(str[i] != '\0' && str[i] != '\n') i++;
    return i;
}

void strReverse(char str[]) {
    int len = strLength(str);
    for(int i = len - 1; i >= 0; i--)
        printf("%c", str[i]);
    printf("\n");
}

void strCopy(char src[], char dest[]) {
    int i = 0;
    while(src[i]) {
        dest[i] = src[i];
        i++;
    }
    dest[i] = '\0';
}

```

```

void strConcat(char s1[], char s2[]) {
    int i = strLength(s1);
    s1[i++] = ' ';
    int j = 0;
    while(s2[j]) {
        s1[i++] = s2[j++];
    }
    s1[i] = '\0';
}

int main() {
    char s[100], s1[100], s2[100];
    printf("Enter string: ");
    fgets(s, sizeof(s), stdin);

    printf("Length = %d\n", strLength(s));
    printf("Reverse = ");
    strReverse(s);

    strCopy(s, s1);
    printf("Copied string: %s\n", s1);

    printf("Enter second string: ");
    fgets(s2, sizeof(s2), stdin);
    strConcat(s, s2);
    printf("Concatenated: %s\n", s);
    return 0;
}

```

10. Write a program to check whether a string is a palindrome.

Algorithm:

1. Start
2. Input string
3. Reverse and compare with original
4. If match, it's a palindrome
5. Print result
6. Stop

Code:

```
#include <stdio.h>
```

```
int isPalindrome(char *str) {
    int len = 0;
    while(str[len] && str[len] != '\n') len++;

    for(int i = 0; i < len / 2; i++) {
        if(str[i] != str[len - i - 1])
            return 0;
    }
    return 1;
}

int main() {
    char str[100];
    printf("Enter string: ");
    fgets(str, sizeof(str), stdin);

    if(isPalindrome(str))
        printf("Palindrome\n");
    else
        printf("Not Palindrome\n");

    return 0;
}
```